

Project scope:

The project is an automated web application penetration testing framework, this is a tool meant to automatically scan web applications for common security vulnerabilities. The tool will test for vulnerabilities such as cross site scripting or SQL injection. The problem this project solves is mainly time constraints, as checking every point of a web application manually would take far too much time. Therefore, this framework will seek to streamline that, doing so automatically, allowing the user to focus on more complex tasks, like solving those vulnerabilities. This project will allow me to learn cybersecurity concepts while providing tangible proof of my coding ability and project management. The framework should crawl through a web application, finding every aspect of the application, test said application for a number of common security risks, and return a report listing them out.

Out of scope: This section is used for what may not be possible in the timeframe for this project.

- Implementing every type of vulnerability that could be incorporated.
- Detecting advanced security risks.
- Creating my own intentionally vulnerable web applications to test with this framework
- Web dashboard that could include previous results
- AI analysis on how to fix the found risks.

Minimum Viable Product:

This section is meant to define the minimum requirements for the project to be considered completed.

1. The framework can accept a website URL

2. Crawl the intended website
3. Find pages/forms/parameters
4. Perform 3-5 security checks
5. Identify and structure findings
6. Generate an HTML report

Requirements:

The framework is meant to take a user's input of a website URL, for the purposes of the project in the given time frame will be intentionally vulnerable web applications like OWASP (Open Worldwide Application Security Project) Juice Shop and DVWA (**Damn Vulnerable Web Application**). The applications should be properly formatted, and able to be connected to by the program. The framework's output should be an HTML report detailing every aspect of the application that was discovered during its crawl, each security risk detected by the program including: risk severity, where the risk was found, a description of what the risk is, and a recommendation of how to fix said risk. The report should be able to open in a browser web page showing the results neatly and clearly.

Vulnerabilities: The framework is meant to test applications for web vulnerabilities, as such approximately 3-5 should be chosen to test for the minimum viable product, the vulnerabilities chosen are:

- Missing or improperly configured security headers
- Cross-Site Scripting (XSS)
- SQL Injection (SQLi)
- Insecure Direct Object References
- Sensitive Data Exposure

Notes for basic concepts related to project

- **HTTP (HyperText Transfer Protocol):** HTTP is the communication protocol that allows a browser or program to communicate with a web server. It defines how information is sent between the client (your browser/tool) and the server. HTTPS is the secure version of HTTP because it encrypts the communication.
- **URLs (Uniform Resource Locators):** A URL is the address used to locate a resource on a website. For example, <https://example.com/login> tells your tool to use HTTPS to communicate with the example.com server and access the [/login](https://example.com/login) resource.
- **Requests:** An HTTP request is a message sent **from the client to the server** asking it to perform an action or provide information. Common request methods include [GET](#) (retrieve information), [POST](#) (send information), [PUT](#) (update information), and [DELETE](#) (remove information).
- **Responses:** An HTTP response is the server's reply to a request. It usually contains a **status code**, headers, and sometimes content. For example, [200 OK](#) means the request succeeded, while [404 Not Found](#) means the requested resource could not be found.
- **Endpoints:** An endpoint is a specific location on a web application that can receive requests. For example, [/login](#), [/users](#), and [/api/products](#) could be endpoints. The framework needs to discover these locations so it knows what parts of a website can be tested.
- **Parameters:** Parameters are pieces of information provided to an endpoint to tell the application what data to use. For example, <https://example.com/search?q=computer> contains

the parameter `q` with the value `computer`. Parameters can appear in URLs, forms, cookies, or request bodies.

- **Vulnerabilities:** A vulnerability is a weakness in a website or application that could allow an attacker to perform an unintended action or access information they should not have. Examples include **SQL injection, Cross-Site Scripting (XSS), broken access control, and insecure configurations**. The framework will examine requests, responses, endpoints, and parameters to look for signs of these weaknesses.

Notes for tools related to project

- **Requests:** Python's `requests` library allows the framework to send HTTP requests to a website and receive its responses. It is useful for basic tasks such as visiting URLs, submitting forms, sending parameters, and checking HTTP status codes.
- **BeautifulSoup:** BeautifulSoup is a Python library used to read and analyze HTML received from a website. The framework can use it to find links, forms, input fields, and other elements on a webpage during crawling and reconnaissance.
- **Selenium/Playwright:** Selenium and Playwright automate real web browsers. They are useful when a website relies heavily on JavaScript, dynamic content, buttons, or interactive forms that a simple `requests` call cannot properly access. The framework could use them to interact with these pages and discover additional endpoints or functionality.
- **Jinja2:** Jinja2 is a Python templating engine that can generate HTML pages from data. In the project, it can take the

vulnerabilities and other findings discovered by the scanner and insert them into a professional HTML security report automatically.

How everything should interact

The components should work together as a pipeline. The **crawler** starts with the website URL and discovers pages, links, forms, endpoints, and parameters. It passes this information to the **vulnerability scanners**, which test the discovered endpoints and parameters for your selected security issues. The scanners send their results to the **results manager**, which organizes, removes duplicates, categorizes, and assigns severity to the findings. Finally, the **report generator** takes the organized findings from the results manager and uses **Jinja2** to create a professional HTML report.

Each component should have **one main responsibility**. The crawler should focus on discovery, scanners on detection, the results manager on organizing findings, and the report generator on presenting the results. This makes the framework easier to test, debug, and expand later